Compliance and Secure Coding Practices Report: NeuroShell: An Autonomous Natural-Language-Driven Orchestration Framework for Offensive Security Operations

NeuroShell Core Team — Group R26-CS-018 Department of Computer Systems Engineering
MSc Cybersecurity Research Project
Supervisor: Prof. Harinda Fernando

Abstract

This report documents the compliance and secure coding practices applied throughout the development of NeuroShell, a four-component framework (Intent Recognition Engine, Dynamic Planner/RAG, Adaptive Execution and Error Recovery Engine, and AI-driven Vulnerability Analysis Engine) for authorised offensive-security operations. It states the compliance standards adopted, the secure coding standards followed, how both were embedded in the Secure Development Lifecycle (SDLC), the tools used, the code review and audit process, the ethical safeguards, and the principal challenges and their resolutions. All statements are grounded in the project's verification artefacts (506 automated tests, 76% statement coverage, CI security scanning, and the append-only audit log).

Index Terms

Compliance, secure coding, GDPR, ISO/IEC 27001, OWASP, CERT, SSDF, secure SDLC, prompt injection, RBAC, audit logging.

I. Compliance Standards

A. Applicable and non-applicable regimes

NeuroShell is an authorised-assessment tool operated on private infrastructure under a defined engagement scope. It does not process health information, so *HIPAA* is not applicable. It processes operator commands and the technical artefacts of an assessment rather than personal data of data subjects, so the applicable regimes are data protection law as far as operator records are concerned, and information security management for the software itself. table I maps each standard to its applicability and to the mechanism that implements it.

Standard	Applicability	How it was integrated
GDPR (EU 2016/679)	Partial — operator audit records	Data minimisation and pseudonymisation: audit log stores a SHA-256 digest, never the raw command; purpose and storage limitation enforced by policy
ISO/IEC 27001:2022	Applicable — ISMS for the software	Access control (A.9), operations security (A.12), secure development (A.14) reflected in RBAC, logging, and CI controls
HIPAA	Not applicable	No protected health information is collected or processed

TABLE I
Compliance standards mapped to their project implementation.

B. GDPR integration

The GDPR principles were applied to the one place where operator input is persisted: the audit log. Raw natural-language commands are never stored; a SHA-256 digest is retained instead (listing 1), which satisfies data minimisation (Art. 5(1)(c)) and the security-of-processing obligation (Art. 32) while keeping the record useful for accountability. Retention is bounded by policy, and the audit store is append-only, giving integrity without exposing input content.

C. ISO/IEC 27001 integration

ISO/IEC 27001 provided the control framework for the software itself. Least-privilege access control is implemented by the role hierarchy (viewer → analyst → operator → admin; listing 2), where an unknown role is treated as the most restrictive. Operations security is evidenced by structured logging, health checks, and an append-only audit trail, and secure development is evidenced by the CI gates described in section V.

II. Secure Coding Standards

A. OWASP Secure Coding Practices and Top 10

The OWASP Secure Coding Practices and the OWASP Top 10 (2021) guided the input-handling and access-control code, and the OWASP Top 10 for LLM Applications (2025) guided the language-model layer. table II maps the highest-risk categories to concrete controls.

Risk category	Control in NeuroShell
LLM01 Prompt injection	Three-layer AdversarialDetector (lexical, structural, scoring) with auto-block on critical
A03 Injection	RegexValidator shell-metacharacter scan and parameterised SQLite statements
A01 Broken access control	RBACGuard enforces a pre/post-inference role matrix; least privilege by default
A02 Cryptographic failures	SHA-256 for audit integrity; HS256 JWT with short 30-minute expiry
A09 Logging failures	Structured logging plus append-only, hashed audit records

TABLE II
OWASP risk categories mapped to NeuroShell controls.

B. CERT and ISO/IEC 27001 coding expectations

Following the SEI CERT secure-coding guidance, the codebase avoids dynamic execution constructs (`eval`/`exec`), validates every external field with explicit regular expressions and Pydantic schemas, bounds resources through rate limiting, and handles failures without leaking internal state. Metadata from the language model is never trusted: every entity it produces (IP addresses, CIDR blocks, CVE identifiers, ports, URLs) is re-verified deterministically before use.

Listing 1. Privacy-preserving, parameterised audit write (`audit_logger.py`).

```
def hash_raw_input(raw_input: str) -> str:
    return hashlib.sha256(raw_input.encode("utf-8")).hexdigest()
...
cursor.execute("INSERT INTO audit_log (...) VALUES (?, ?, ?, ...);",
               (ts, session_id, raw_hash, intent_summary, ...))
```

Listing 2. Least-privilege role gating (`rbac_policy.py`).

```
INTENT_ROLE_POLICY = {
    IntentType.EXPLOITATION: ["operator", "admin"],
    IntentType.NETWORK_SCAN: ["analyst", "operator", "admin"], ...}
# Unknown roles are treated as viewer (most restrictive).
def is_role_permitted(role, intent):
    return role in INTENT_ROLE_POLICY.get(intent, [])
```

III. Integration of the Secure Development Lifecycle

Security and compliance were treated as first-class requirements rather than a post-hoc review. table III summarises the emphasis on each SDLC phase.

Phase	Security and compliance emphasis
Requirements	Safety invariant and functional/non-functional security requirements (RBAC roles, scope containment, rate limits) specified up front
Design	Defence in depth: independent deterministic validators that cannot be bypassed by the model; contract schema fixed before inference
Implementation	Type-annotated modules, input normalisation, no dynamic execution, secrets kept out of source
Testing	506 deterministic tests (434 unit, 72 integration); adversarial, injection, and RBAC negative paths
Deployment	Pinned dependencies, minimal container images, environment-injected secrets, health checks
Maintenance	CI gates (lint, type check, test, Bandit security scan) on every pull request

TABLE III
Security and compliance emphasis across the SDLC phases.

IV. Tools for Security and Compliance

A layered toolset supports both security analysis and compliance management (table IV). Static application security testing is performed by Bandit, which scans the codebase for common Python weaknesses and is run as a dedicated CI job. Linting, formatting, and type checking (Ruff, Black, Mypy) keep the code consistent and catch defects early, while Pytest with Coverage measures the verification gate. GitHub Actions orchestrates the pipeline, and Docker with the Kong API gateway provide reproducible, isolated deployment.

Tool	Purpose	Role in the project
Bandit	SAST	Security scan in CI (`-x ./tests,./infra`)
Ruff / Black	Linting / format	Style and defect drift control in CI
Mypy	Type checking	Static type verification of services
Pytest + Coverage	Testing	506 tests passing, 76% coverage
GitHub Actions	Orchestration	Lint, typecheck, test, security-scan, build
Docker / Kong	Isolation / gateway	Reproducible services and routed edge

TABLE IV
SECURITY AND COMPLIANCE TOOLING.

V. Code Review and Audit Process

All changes reach the protected branches only through pull requests, and the CI pipeline enforces the review gate: the pull request cannot merge unless the lint, type-check, test, and security-scan jobs pass, and the container build is also exercised for reviewed changes. Reviewers check both correctness and security-relevant behaviour, with particular attention to the deterministic validators, the RBAC matrix, and any code touching persisted data. Feedback is incorporated in subsequent commits on the same branch, and additional regression tests are added when a review uncovers a weakness. The audit dimension is satisfied at runtime by an append-only SQLite trail: every enforcement decision is recorded with its class, severity, and action, raw input is hashed, and unresolved findings enter an escalation queue that an administrator resolves through a dedicated endpoint. A Quality Assurance report, produced from the test, coverage, and static-analysis runs, captures the outcome and feeds the next development cycle.

VI. Ethical and Compliance Considerations

NeuroShell is a dual-use offensive-security tool, so its ethical guardrails are enforced in code rather than left to operator discretion. Targets are constrained to the RFC 1918 private-address space and the configured engagement scope, so the framework cannot be aimed at systems outside the authorised boundary. High-impact operations are gated by the role hierarchy, and ambiguous or contradictory requests are refused or routed to a human reviewer instead of being executed automatically. No personal data of data subjects is collected, operator input is pseudonymised before storage, and all usage is subject to the project's academic-use and responsible-disclosure commitments. AI assistance in the project is documented separately in the AI-use disclosure appendix of the thesis.

VII. Challenges and Solutions

Implementing secure coding and compliance in an LLM-driven system presented several recurring challenges; table V records each and how it was addressed.

Challenge	Solution adopted
Prompt injection and model hallucination	Deterministic validation stack and adversarial detector that override the model; never trust model output
Balancing security strictness against usability	Configurable enforcement modes (research vs. strict) and a warn/block distinction per finding class
Privacy of operator input while keeping auditability	Store SHA-256 digests only, in an append-only log, and never persist raw commands
Ensuring consistent checks across a multi-service codebase	Central CI gates (lint, type, test, Bandit) and a shared, typed contract between components
Verifying safety without a live model	Deterministic test fixtures that mock the inference engine, so the safety core is tested independently

TABLE V
CHALLENGES ENCOUNTERED AND THE SOLUTIONS ADOPTED.

Conclusion

Compliance and secure coding were embedded throughout NeuroShell rather than added at the end. GDPR principles shape the privacy-preserving audit log, ISO/IEC 27001 controls the access and operations posture, and the OWASP, CERT, and NIST SSDF guidance directs the input handling, LLM hardening, and CI verification gates. The result is a framework whose safety-critical behaviour is deterministic, testable, and auditable, and whose residual risks are documented rather than hidden.

References

[1] OWASP Foundation, *OWASP Secure Coding Practices Quick Reference Guide*, 2010.
[2] OWASP Foundation, *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*, 2021.
[3] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*, 2025.
[4] CERT Division, Software Engineering Institute, Carnegie Mellon University, *SEI CERT Coding Standards*, 2023.
[5] International Organization for Standardization, *ISO/IEC 27001:2022 — Information security, cybersecurity and privacy protection — Information security management systems — Requirements*, 2022.
[6] European Parliament and Council, *Regulation (EU) 2016/679 (General Data Protection Regulation)*, 2016.
[7] National Institute of Standards and Technology, *NIST SP 800-218: Secure Software Development Framework (SSDF) Version 1.1*, 2022.
[8] PyCQA, *Bandit: A security linter for Python*. [Online]. Available: https://bandit.readthedocs.io